Take-home assignment: start here

Thank you for your time in the first conversation. The next step is a take-home assignment. There are three to choose from, and you do **one**. Each is modelled on a real kind of project our engineers deliver for customers in Southeast Asia. All companies, people and data in them are fictional.

The three options

	Assignment	Level	What it emphasises
A1	Document Intelligence: a question-answering assistant over an annual report	Easy	Sound RAG fundamentals, evaluation, honest answers
A2	Transaction Reconciliation: supplier invoices against an ERP export	Medium	Data integrity, deterministic logic, auditability
A3	Product Discovery: a sales assistant over a product catalog website	Hard	End-to-end design: data, search, agents, pricing

Pick the one where you can do your best work. A harder assignment earns no extra points; an excellent A1 beats an average A3. Harder assignments only give more room to show advanced skills.

Time

- Plan for about **6 hours** of focused work. Please do not spend much more; good judgment about scope is part of what we look at.
- You have **5 calendar days** from the day you receive this. The exact deadline (18:00 your local time) is in our email.
- If you need more time for any reason, including a holiday, tell us. Asking is fine.

What to submit

1. A Git repository link (GitHub or GitLab; private is fine if you give us access). Never commit API keys.
2. A **README** with: how to run it in 5 commands or fewer, an architecture overview, key decisions and trade-offs, known limitations, and what you would do with another week.
3. Your **evaluation results** as described in your brief, including cases where your system fails.
4. A **one-page customer summary** (PDF or Markdown) written for the customer's business owner, not an engineer: what you built, how well it works, what it cannot do yet, and what you recommend next.
5. One short README paragraph on **how you used AI tools** while building.

Submit by replying to the email we sent you, with your repository link and your one-page summary.

Rules

- Any language, framework, model provider or database. Free tiers and local models are fine.
- AI coding assistants are allowed. You must be able to explain and change any part of your code in the interview.
- If model API costs are a barrier, tell us and we will give you a capped API key.
- Use the data only for this assignment.
- Questions are welcome: reply to our email and we answer within one business day. Asking good clarifying questions is part of the job.

What happens after you submit

Round	Length	What happens
Demo and deep dive	60 min	You demo your work. We run it on inputs you have not seen. We discuss design, testing, failures and production readiness.
Customer simulation	40 min	A role-played discovery call, a short solution pitch and a conversation with a sceptical stakeholder.
Live engineering	45 min	A small debugging and diagnosis exercise. Bring your usual tools.

Keep your solution runnable for the demo round: we will ask you to process a new input file live, without changing code first.

How we evaluate

Every candidate is scored on the same seven areas, whichever assignment they pick: technical correctness, problem solving and judgment, engineering quality, AI and data evaluation, communication, business and pre-sales thinking, and ownership. We care more about evidence that your system works, and honesty about where it does not, than about polish.

A1 Document Intelligence

Level: Easy. Plan for about 6 hours.

The customer

Kirana Logistics Group operates warehouses and trucking across Thailand, Indonesia, Vietnam and Malaysia. Its investor relations team answers the same analyst questions about the annual report every week. They want an assistant that answers questions from the report, shows where each answer comes from, and never makes things up.

What you get

- **Kirana_Logistics_Annual_Report_2025.pdf** (13 pages). It contains prose, tables, charts, an infographic, a diagram and a scanned page. As in most real reports, some facts appear only inside charts or images.

What to build (must have)

1. An **ingestion pipeline** that processes the whole PDF. If you choose not to handle some content type, say so and explain why.
2. A way to **ask questions**: CLI, API or simple web UI.
3. Answers that **cite their source**: page number at minimum, ideally the table, figure or passage.
4. **Honest refusals.** If the document does not support an answer, say so instead of guessing.
5. An **evaluation set** of 15 to 20 questions you write yourself, each with an expected answer and the page that supports it. Cover different content types, plus at least three questions the document cannot answer.
6. **Evaluation results** that measure retrieval (did the right evidence come back?) separately from answer quality (was the answer right?). Show the failures and what you think causes them.
7. The pipeline must work on a **new PDF without code changes**. In the demo round we will give you a second document from the same company.

Nice to have (pick any, or none)

- Hybrid search or reranking, with evidence that it helps
- Questions across several documents
- Cost and latency per question
- Conversation follow-ups
- A clean UI that shows the cited page

What we look for

Area	Strong submissions
Ingestion	Choose the right method per content type: text extraction, table parsing, OCR or a vision model. Explain the choice.
Chunking and retrieval	Can justify chunk size, overlap, metadata and top-k for this document.
Grounding	Verifiable citations; clear refusals when evidence is missing; conflicting statements surfaced, not hidden.

Area	Strong submissions
Evaluation	A repeatable test set, retrieval and answer quality measured separately, honest failure analysis.
Communication	README and customer summary a busy reader understands in five minutes.

Tip: start with a baseline that works end to end, measure it, then improve the weakest part. Show us that loop.

A2 Transaction Reconciliation

Level: Medium. Plan for about 6 hours.

The customer

PT Nusantara Retail Group runs a chain of stores in Indonesia. Every month its accounts payable team reconciles supplier invoices against what was booked in the ERP. Three people spend two days on it, and mistakes lead to overpayments. They want a system that does the matching, explains every difference, and leaves the final decision to a person.

What you get

- **Supplier_Invoices_PT_Prima_Kertas_Sep2026.pdf**: the September batch from one supplier, as received.
- **ERP_AP_Export_Sep2026.xlsx**: the company's records, exported from its ERP (read all sheets).
- The business rules below. Treat them as the customer's agreed specification.

Both files are realistic. Expect formatting differences in identifiers, dates and numbers, and expect that either side can contain errors. In production the customer has more than 40 suppliers, each with its own invoice layout. Your sample comes from one supplier.

Business rules

#	Rule
R1	Line amount = quantity x unit price x (1 - discount % / 100), rounded half-up to whole rupiah.
R2	Subtotal = sum of line amounts.
R3	DPP Nilai Lain (other tax base) = subtotal x 11/12, rounded half-up to whole rupiah.
R4	PPN (VAT) = DPP x 12%, rounded half-up to whole rupiah. Total = subtotal + PPN.
R5	A difference of up to IDR 10 on the total, with no line-level differences, is rounding and counts as matched within tolerance.
R6	Credit notes reduce the amount owed and are reconciled like invoices.
R7	Never correct a figure automatically. If a document's own numbers do not add up, or key references disagree, flag it for review.
R8	Use one status per document: MATCHED, MATCHED_WITHIN_TOLERANCE, AMOUNT_MISMATCH, MISSING_IN_ERP, MISSING_IN_PDF, DUPLICATE_IN_ERP, DUPLICATE_IN_PDF or NEEDS_REVIEW. Add reason codes of your own design.

The tax rules are simplified for this exercise. They follow the Indonesian approach in force since 2025 (12% applied to an 11/12 base, an effective 11%) but are not tax advice.

What to build (must have)

1. Extraction of every document in the PDF into structured records, including pages that are hard to read.
2. Normalisation and matching between PDF documents and ERP entries, at document and line level.
3. All arithmetic and matching done in **deterministic code**. Use an LLM only where it adds value, and explain where that is.

4. A **reconciliation report** (CSV or Excel): one row per document with PDF total, ERP total, difference, status, reason codes, a plain-language explanation and a suggested action.
5. A **line-level discrepancy list** and **batch control totals** showing that the overall difference between the two files is fully explained.
6. An **audit trail**: every figure traceable to a PDF page or an ERP sheet and row.
7. Tests for your matching and calculation logic.
8. It must run on a **new batch without code changes**. In the demo round we will give you the October files.

Nice to have

- A review queue or simple UI for the AP team
- Confidence scores and how you would use them
- How you would onboard a new supplier layout safely
- Cost and runtime per batch

What we look for

Area	Strong submissions
Extraction	Reliable structured records from every page, with checks that catch extraction errors.
Judgment	Knows which parts need an LLM and which need plain code. Separates a real discrepancy from formatting or rounding.
Correctness	Right status and cause for each document; totals reconcile across the whole batch.
Auditability	A finance reviewer can verify any result in under a minute.
Communication	Explanations a finance clerk understands; a customer summary an AP manager would forward to their CFO.

A3 Product Discovery

Level: Hard. Plan for about 6 hours; scope it.

The customer

PaperLane sells paper and flyer printing to offices in Bangkok. Most orders start as a chat message such as "We need letter-size flyers, one side, colour, under 250 a pack, 20 packs." Sales staff answer by hand. PaperLane wants an assistant that understands these requests, finds the right products and gives an accurate quote.

What you get

- The PaperLane catalog website as **PaperLane_site_v1.zip**. To run it: unzip, run `python -m http.server 8000` in the unzipped folder, and open `http://localhost:8000/paperlane/`. It must be served over HTTP; opening the files from disk will not load volume prices. If our email includes a hosted URL, it is an identical copy.
- Permission: you may crawl the site automatically, up to 2 requests per second. Treat it like a real website and inspect it carefully; not all content is in the initial HTML.
- The pricing rules below. Treat them as the customer's agreed specification.

Pricing rules

#	Rule
PR1	All prices on the site exclude 7% VAT and are per pack (pack size varies by product).
PR2	Volume pricing applies per item, based on the number of packs of that item: use the highest tier whose minimum quantity is met.
PR3	A sale price replaces both the list price and volume pricing. They cannot be combined.
PR4	Subtotal = sum of unit price x quantity. Delivery is free when the subtotal is THB 3,000 or more; otherwise THB 150 per order.
PR5	VAT = 7% of (subtotal + delivery), rounded half-up to 2 decimals. Total = subtotal + delivery + VAT.
PR6	When a customer gives a maximum price per pack, compare it with the unit price they would pay at the requested quantity, before VAT.
PR7	Budgets are assumed to exclude VAT unless the customer says otherwise. Say which assumption you used.
PR8	Out-of-stock and discontinued items must not be quoted as available.

Weights appear in gsm and in US bond pounds. Use: 20 lb = 75 gsm, 24 lb = 90 gsm, 28 lb = 105 gsm, 32 lb = 120 gsm. Customers describe sizes and options informally.

What to build (must have)

1. An **ingestion pipeline** that crawls the site into a database of normalised product records, including variants, volume pricing, stock status and provenance (source URL and time).
2. A **schema** you can defend for size, weight, colour, finish, sides, colour mode, pack size, prices and stock.
3. An **assistant** that turns a request into validated search criteria, calls tools, and replies with matching products, why they match, and a full price breakdown.
4. **Pricing calculated by deterministic code**, never by the model.

5. Clarifying questions when a request is ambiguous; a clear "no exact match" with the closest alternatives when nothing fits; no invented products.
6. An **API** (for example POST /chat or /quote) and a simple UI or CLI.
7. A **test set** of at least 10 requests with expected products and totals, and your results.
8. A **refresh** path. In the demo round we will give you an updated version of the site and ask you to re-ingest it without code changes.

Nice to have

- Hybrid search (structured filters plus semantic matching)
- Multi-item quotes and conversation memory
- Defences against untrusted text in scraped content
- Logging and metrics you would watch in production
- Notes on deploying it behind a LINE or WhatsApp chat

What we look for

Area	Strong submissions
Data	Complete, correctly typed records, including what is hidden in variants or loaded by script.
Search	Hard constraints as filters, softer wishes as ranking; unit and size normalisation.
Agent design	Clear tools and validation; the model decides intent, code decides facts and prices.
Correctness	Right products and totals to the satang on unseen requests, including edge cases.
Business sense	Replies a salesperson would be proud to send; honest when nothing fits.